Artificial Intelligence

BS (CS) _SPRING_2026

Lab_09 Manual



Learning Objectives:

1. To Understand Adversarial Search
2. To understand the concept of adversarial (multi-agent) search.
3. To implement the Minimax algorithm for decision-making.
4. To analyze how optimal strategies are formed in competitive environments.
5. To study the performance and limitations of Minimax.

Introduction to Adversarial Search

Adversarial search is a type of search used in Artificial Intelligence where multiple agents interact in a competitive environment. Unlike single-agent search, where one agent tries to reach a goal, adversarial search involves **opposing agents** whose goals conflict.

This is commonly seen in **game-playing scenarios** such as chess, tic-tac-toe, and checkers.

Key Characteristics:

- Multiple agents (players)
- Each agent acts against others
- Decisions depend on opponent actions
- Environment is called a **multi-agent environment**

Core Idea:

The AI assumes:

"The opponent will always make the best possible move."

Minimax Algorithm

The **Minimax Algorithm** is a fundamental adversarial search algorithm used in turn-based games.

- **MAX (AI Player)** → tries to maximize the score
- **MIN (Opponent)** → tries to minimize the score

The algorithm explores all possible game states and selects the move that leads to the best outcome under worst-case assumptions.

Key Concepts:

1. Game Tree

A tree structure representing all possible moves:

- Nodes → game states
- Edges → actions

2. Terminal State

A state where the game ends:

- Win
- Loss
- Draw

3. Utility Function

Assigns exact values:

- Win = +1
- Loss = -1
- Draw = 0

4. Heuristic Function

Used when full search is not possible:

- Estimates how good a state is

Step-by-Step Procedure

Step 1: Initialize State

Start with the current game configuration.

Step 2: Generate Moves (MOVE_GEN)

Find all valid moves from current state.

Step 3: Apply Move

Create a new state after applying a move.

Step 4: Terminal Test

Check if the game is over.

Step 5: Evaluate State

Assign a score using utility or heuristic function.

Step 6: Recursive Minimax

- MAX → chooses highest value
- MIN → chooses lowest value

Step 7: Choose Best Move

Select move with the best guaranteed outcome.

Pseudocode

```
function MINIMAX(state, depth, isMaxPlayer):

    if TERMINAL_TEST(state) or depth == 0:
        return EVALUATE(state)

    if isMaxPlayer:
        bestValue = -∞
        for move in MOVE_GEN(state):
            value = MINIMAX(APPLY_MOVE(state, move), depth-1, False)
            bestValue = max(bestValue, value)
        return bestValue

    else:
        bestValue = +∞
        for move in MOVE_GEN(state):
            value = MINIMAX(APPLY_MOVE(state, move), depth-1, True)
            bestValue = min(bestValue, value)
        return bestValue
```

Implementation

```
def minimax(state, is_max):
    if is_terminal(state):
        return evaluate(state)

    if is_max:
        best = float('-inf')
        for move in get_moves(state):
            val = minimax(apply_move(state, move), False)
            best = max(best, val)
        return best
    else:
        best = float('inf')
        for move in get_moves(state):
            val = minimax(apply_move(state, move), True)
            best = min(best, val)
        return best
```

Example: Tic-Tac-Toe (Step-by-Step)

Players:

- **MAX → X (AI)**
- **MIN → O (Opponent)**

Initial Board

X | O | X
O | X | -
- | - | O

Empty cells = possible moves

Step 1: Generate Moves

Possible positions:

- (2,0)
- (2,1)
- (1,2)

Step 2: Build Game Tree

Example branch:

Move at (2,0)

X | O | X
O | X | -
X | - | O

Step 3: Evaluate Outcomes

Explore all future moves until terminal states:

Branch Values:

Move (2,0) → +1 (Win)
Move (2,1) → 0 (Draw)
Move (1,2) → -1 (Loss)

Step 4: Backtracking (Minimax)

- MIN tries to reduce score
- MAX selects best guaranteed result

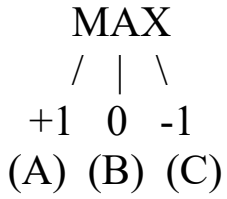
Step 5: Final Decision

Best Move = (2,0)

Because:

+1 > 0 > -1

Visualization of Game Tree



MAX selects the branch with **highest value → +1**

Complexity Analysis

Time Complexity

$O(b^d)$

- **b** = branching factor
- **d** = depth

Space Complexity

$O(d)$ (due to recursion stack)

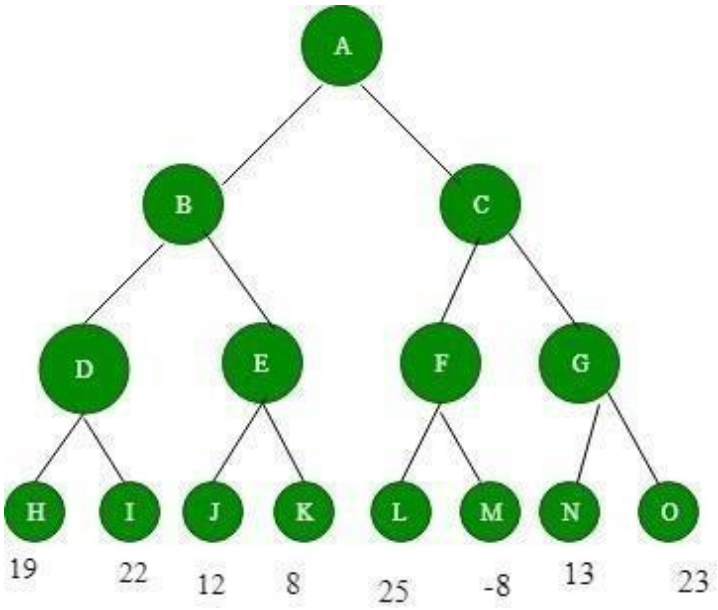
Minimax algorithm uses two functions

MOVE GEN: It generates all the possible moves that can be generated from the current position.

STATIC EVALUATION: It returns a value depending upon the goodness from the viewpoint of two-player.

This algorithm is a two-player game, so we call the first player as PLAYER1 and second player as PLAYER2. The value of each node is backed-up from its children. For PLAYER1 the backed-up value is the maximum value of its children and for PLAYER2 the backed-up value is the minimum value of its children. It provides most promising move to PLAYER1, assuming that the PLAYER2 has made the best move. It is a recursive algorithm, as same procedure occurs at each level.

Before Backing-up values



After Backing-up values

